

CAPÍTULO 10

Árvores Gramaticais

No capítulo anterior vimos como gerar palavras a partir de uma gramática livre de contexto, por derivação. Neste capítulo consideramos outra maneira pela qual uma gramática livre de contexto gera palavras: as árvores gramaticais. Esta noção tem origem na necessidade de diagramar a análise sintática de uma sentença em uma língua natural, como o português ou o japonês.

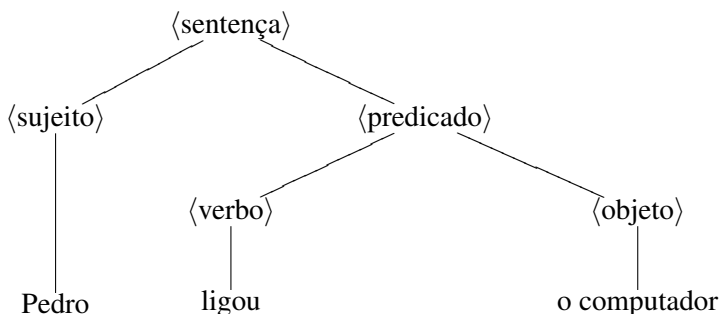
1. Análise Sintática e línguas naturais

Até agora as gramáticas formais que introduzimos têm servido basicamente para gerar palavras que pertencem a uma dada linguagem. Já a gramática da língua portuguesa não serve apenas para gerar frases com sintaxe correta, mas também para analisar a estrutura de uma frase e determinar o seu significado. A isso se chama análise sintática. Descobrir quem fez o quê, e a quem, é uma questão de identificar sujeito, verbo e objetos de uma frase. Entretanto, a noção de derivação não é uma ferramenta adequada para realizar a análise sintática em uma gramática; para isto precisamos das árvores gramaticais.

Vejamos como usar uma árvore para diagramar a análise sintática da frase

Pedro ligou o computador.

O sujeito da sentença é *Pedro* e o predicado é *ligou o computador*. Por sua vez o predicado pode ser decomposto no verbo *ligou* seguido do objeto direto, *o computador*. Esta análise da frase pode ser diagramada em uma árvore como a da figura abaixo.



Note que as palavras sentença, sujeito, predicado, verbo e objeto direto aparecem entre $\langle \ \rangle$. Fizemos isto para diferenciar o conceito, da palavra da língua portuguesa que o denota. Em outras palavras, $\langle \text{sujeito} \rangle$ indica que na gramática da língua portuguesa há uma variável chamada de sujeito.

Existe uma estreita relação entre a estrutura da árvore acima e as regras da gramática da língua portuguesa. Lendo esta árvore de cima para baixo, cada bifurcação corresponde a uma regra da gramática portuguesa. As regras que aparecem na árvore da figura acima são as seguintes:

$\langle \text{sentença} \rangle \rightarrow \langle \text{sujeito} \rangle \langle \text{predicado} \rangle$
 $\langle \text{predicado} \rangle \rightarrow \langle \text{verbo} \rangle \langle \text{objeto direto} \rangle$
 $\langle \text{sujeito} \rangle \rightarrow \text{Pedro}$
 $\langle \text{verbo} \rangle \rightarrow \text{ligou}$
 $\langle \text{objeto direto} \rangle \rightarrow \text{o computador.}$

Assim podemos usar esta árvore para derivar a frase “Pedro ligou o computador” a partir das regras acima:

$\langle \text{sentença} \rangle \Rightarrow \langle \text{sujeito} \rangle \langle \text{predicado} \rangle \Rightarrow \langle \text{sujeito} \rangle \langle \text{verbo} \rangle \langle \text{objeto direto} \rangle \Rightarrow$
 $\text{Pedro} \langle \text{verbo} \rangle \langle \text{objeto direto} \rangle \Rightarrow \text{Pedro ligou} \langle \text{objeto direto} \rangle$
 $\Rightarrow \text{Pedro ligou o computador}$

O principal resultado deste capítulo mostra que existe uma estreita relação entre árvores gramaticais e derivações. Antes, porém, precisamos definir formalmente a noção de árvore gramatical de uma linguagem livre de contexto.

2. Árvores Gramaticais

Nesta seção, além de introduzir formalmente o conceito de árvore gramatical de uma linguagem livre de contexto, estabelecemos a relação entre árvores e derivações.

Lembre-se que uma árvore é um grafo conexo que não tem ciclos. As árvores que usaremos são na verdade grafos orientados. Entretanto, não desenharemos as arestas destas árvores como setas. Em vez disso, indicaremos que o vértice v precede v' desenhando v acima de v' . Além disso suporemos sempre que estamos lidando com árvores que satisfazem as seguintes propriedades:

- há um único vértice, que será chamado de *raiz*, e que não é precedido por nenhum outro vértice;
- os sucessores de um vértice estão totalmente ordenados.

Se v' e v'' são sucessores de v , e se v' precede v'' na ordenação dos vértices, então desenharemos v' à esquerda de v'' . De agora em diante a palavra árvore será usada para designar um grafo com todas as propriedades relacionadas acima.

—————Falta um grafo aqui—————

Um exemplo de árvore com estas propriedades, é a árvore genealógica que descreve os descendentes de uma dada pessoa. Note que, em uma árvore genealógica, os irmãos podem ser naturalmente ordenados pela ordem do nascimento. A terminologia que passamos a descrever é inspirada neste exemplo. Suponhamos que $\mathcal{T}$ é uma árvore e que v' é um vértice de $\mathcal{T}$. Então há um único caminho ligando a raiz a v' . Digamos que este caminho contém um vértice v . Neste caso,

- v é *ascendente* de v' e v' é *descendente* de v ;
- se v e v' estão separados por apenas uma aresta então v é *pai* de v' e v' é *filho* de v ;
- se v' e v'' são filhos de um mesmo pai, então são *irmãos*;
- um vértice sem filhos é chamado de *folha*;
- um vértice que não é uma folha é chamado de *vértice interior*.

A ordenação entre irmãos, que faz parte da definição de árvore, pode ser estendida a uma ordenação de todas as folhas. Para ver como isto é feito, digamos que f' e f'' são duas folhas de uma árvore. Começamos procurando o seu primeiro ascendente comum, que chamaremos de v . Observe que f e f'' têm que ser descendentes de filhos diferentes de v , a não ser que sejam eles próprios filhos de v . Digamos que f é descendente de v' e f'' de v'' . Como v' e v'' são irmãos, sabemos que estão ordenados; digamos que v' precede v'' . Neste caso dizemos que

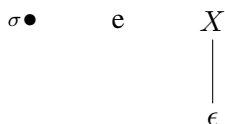
f' precede f'' . Esta é uma ordem total; isto é, todas as folhas de uma tal árvore podem ser escritas em fila, de modo que a seguinte sucede à anterior.

Nenhuma preocupação adicional com esta ordenação é necessária na hora de desenhar uma árvore; basta obedecer à convenção de sempre ordenar os vértices irmãos da esquerda para a direita. Se fizermos isto, as folhas ficarão automaticamente ordenadas da esquerda para a direita.

Seja $\mathcal{G}$ uma gramática livre de contexto com terminais T e variáveis V . De agora em diante estaremos considerando árvores, no sentido acima, cujos vértices estão rotulados por elementos de $T \cup V$. Contudo, não estaremos interessados em todas estas árvores, mas apenas naquelas resultantes de um procedimento recursivo bastante simples, as árvores gramaticais. Como se trata de uma definição recursiva, precisamos estabelecer quem são os átomos da construção, que chamaremos de árvores básicas, e de que maneira podemos combiná-las.

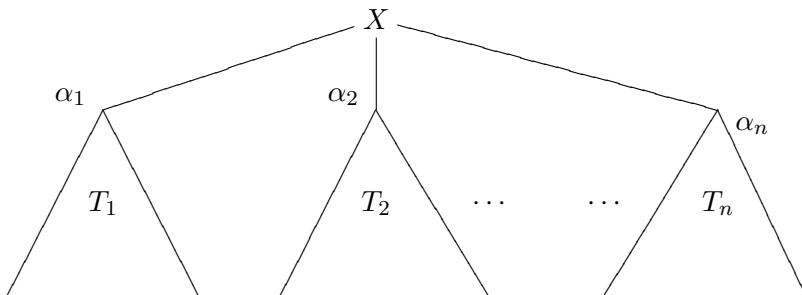
As *árvores gramaticais* são definidas recursivamente da seguinte maneira:

Árvores básicas: se $\sigma \in T$, $X \in V$ e $X \rightarrow \epsilon$ é uma regra de $\mathcal{G}$, então



são árvores gramaticais e suas colheitas são, respectivamente, σ e ϵ .

Regras de combinação: sejam $T_1, \dots, T_n$ árvores gramaticais, e suponhamos que o rótulo da raiz de T_j é α_j . Se $X \rightarrow \alpha_1 \dots \alpha_n$ é uma regra de $\mathcal{G}$, então a árvore T definida por



também é uma árvore gramatical.

Um exemplo simples é a árvore na gramática $\mathcal{G}_{exp}$ (definida na seção 3 do capítulo 7) esboçada abaixo.

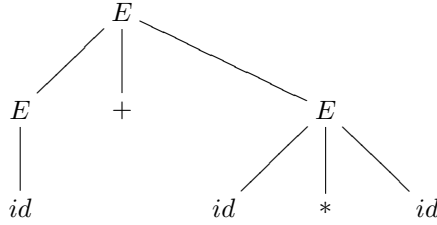


FIGURA 1. Árvore gramatical

Uma consequência muito importante desta definição recursiva, e uma que usaremos em várias oportunidades, é a seguinte. Suponhamos que uma árvore gramatical $\mathcal{T}$ tem um vértice v rotulado por uma variável X . Então podemos trocar toda a parte de $\mathcal{T}$ que descende de v por qualquer outra árvore gramatical cuja raiz seja rotulada por X .

Segue também da definição que os únicos vértices de uma árvore gramatical que podem ser rotulados por elementos de $T \cup \{\epsilon\}$ são as folhas. Portanto, um vértice interior de uma árvore gramatical só pode ser rotulado por uma variável.

Por outro lado, se o vértice v de uma árvore gramatical $\mathcal{T}$ está rotulado por uma variável X , e seus filhos por $\alpha_1, \dots, \alpha_n \in T \cup V$ então $X \rightarrow \alpha_1 \cdots \alpha_n$ tem que ser uma regra da gramática $\mathcal{G}$. Diremos que esta é a regra *associada* ao vértice v . No caso de v ser a raiz, $\mathcal{T}$ é uma X -*árvore*. Caso a árvore gramatical consista apenas de uma folha rotulada por um terminal σ , diremos que se trata de uma σ -*árvore*.

Uma vez tendo introduzido árvores gramaticais temos uma outra maneira de gerar palavras a partir de uma gramática livre de contexto. Para isso, definimos a *colheita* $c(\mathcal{T})$ de uma árvore gramatical $\mathcal{T}$. Como a definição de árvore é recursiva, assim será a definição de colheita. As colheitas das árvores básicas são

$$c(\bullet\sigma) = \sigma \quad \text{e} \quad c\left(\begin{array}{c} X \\ | \\ \epsilon \end{array}\right) = \epsilon$$

Por outro lado sejam $T_1, \dots, T_n$ árvores gramaticais, e suponhamos que o rótulo da raiz de T_j é α_j . Se $X \rightarrow \alpha_1 \cdots \alpha_n$ é uma regra de $\mathcal{G}$, então a árvore T construída de acordo com a *regra de combinação* satisfaz

$$c(T) = c(T_1) \cdot c(T_2) \cdots c(T_n).$$

Como consequência da definição recursiva, temos que a colheita de uma árvore gramatical T é a palavra obtida concatenando-se os rótulos

de todas as folhas de T , da esquerda para a direita. Como as folhas de uma árvore gramatical estão totalmente ordenadas, não há nenhuma ambigüidade nesta maneira de expressar a colheita. Para uma demonstração formal deste fato veja o exercício 1. Portanto, a árvore da figura 1 tem colheita $\text{id} + \text{id} * \text{id}$.

Digamos que w é uma palavra que pode ser derivada em uma gramática livre de contexto $\mathcal{G}$ com símbolo inicial S . Uma *árvore de derivação* para w é uma S -árvore de $\mathcal{G}$ cuja colheita é w . Note que reservamos o nome de árvore de derivação para o caso especial das S -árvores.

3. Colhendo e derivando

No capítulo 7 vimos como gerar uma palavra em terminais a partir do símbolo inicial de uma linguagem livre de contexto por derivação. A noção de colheita de uma árvore gramatical nos dá uma segunda maneira de produzir uma tal palavra. Como seria de esperar, há uma estreita relação entre estes dois métodos, que será discutida em detalhes nesta seção.

Para explicitar a relação entre árvores gramaticais e derivações vamos descrever um algoritmo que constrói uma derivação mais à esquerda de uma palavra a partir de sua árvore gramatical. Heurística-mente falando, o algoritmo desmonta a árvore gramatical da raiz às folhas. Contudo, ao remover a raiz de uma árvore gramatical, não obtemos uma nova árvore gramatical, mas sim uma sequência ordenada de árvores. Isto sugere a seguinte definição. Seja $\mathcal{G}$ uma gramática livre de contexto com terminais T e variáveis V . Se

$$w = \alpha_1 \cdots \alpha_n \in (T \cup V)^*$$

então uma w -floresta $\mathcal{F}$ é uma sequência ordenada $T_1, \dots, T_n$ de árvores gramaticais, onde T_j é uma α_j -árvore. A *colheita da floresta* $\mathcal{F}$ é a concatenação das colheitas de suas árvores; isto é

$$c(\mathcal{F}) = c(T_1) \cdots c(T_n).$$

Podemos agora descrever o algoritmo. Lembre-se que quando dizemos ‘remova a raiz da árvore $\mathcal{T}$ ’ estamos implicitamente assumindo que as arestas incidentes à raiz também estão sendo removidas. Se v é a raiz de uma árvore $\mathcal{T}$ da floresta $\mathcal{F}$, denotaremos por $\mathcal{F} \setminus v$ a floresta obtida quando v é removido de $\mathcal{T}$.

Precisamos considerar, separadamente, o efeito desta construção quando $\mathcal{T}$ é a árvore básica que tem a raiz rotulada pela variável X e uma única folha rotulada por ϵ . Neste caso, quando removemos a raiz sobra apenas um vértice rotulado por ϵ , que não constitui uma árvore

gramatical. Suporemos, então, que remover a raiz de uma tal árvore tem o efeito de apagar toda a árvore.

ALGORITMO 10.1. *Constrói uma derivação a partir de uma árvore gramatical.*

Entrada: uma X -árvore $\mathcal{T}$, onde X é uma variável.

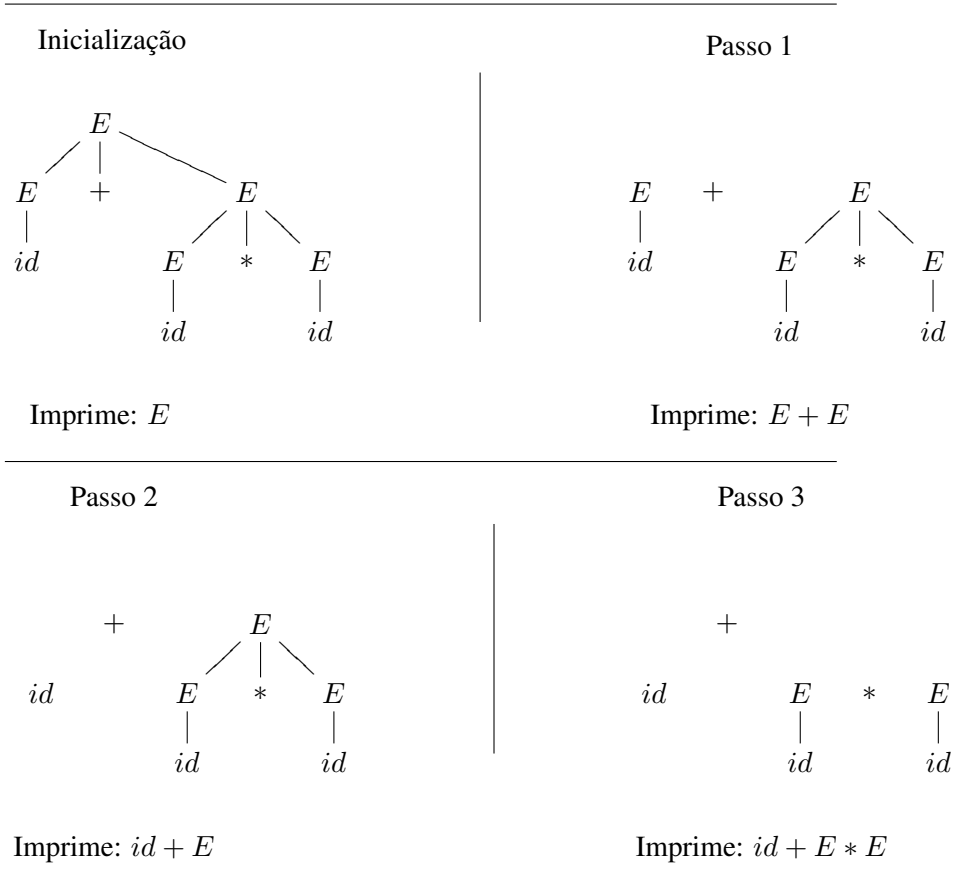
Saída: uma derivação mais à esquerda de $c(\mathcal{T})$.

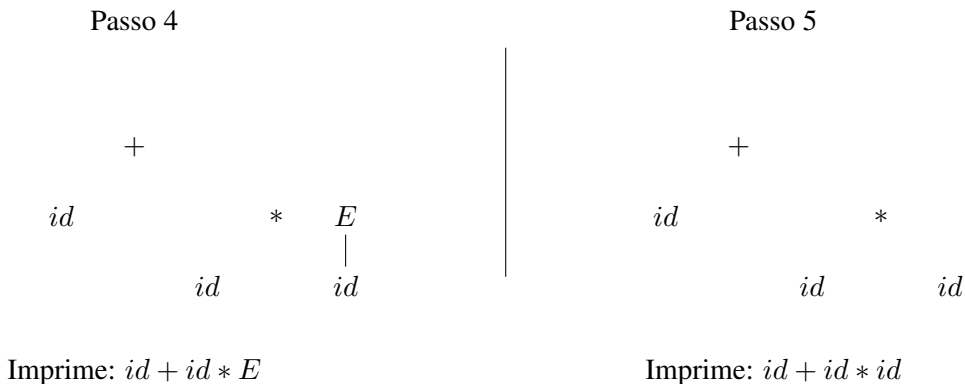
Etapa 1: Inicialize $\mathcal{F}$ com $\mathcal{T}$.

Etapa 2: Se $\mathcal{F}$ é uma w -floresta, então imprima w . Páre se todas as árvores de $\mathcal{F}$ são rotuladas por terminais.

Etapa 3: Seja v a raiz da árvore mais à esquerda de $\mathcal{F}$ que é rotulada por uma variável. Faça $\mathcal{F} = \mathcal{F} \setminus v$ e volte à etapa 2.

À seguir, você encontrará um exemplo da aplicação passo a passo deste algoritmo a uma árvore na gramática $\mathcal{G}_{exp}$.





Resta-nos demonstrar que este algoritmo funciona. Começamos por analisar o que o algoritmo faz quando um de seus passos é executado. Suponhamos que, ao final do k -ésimo passo, temos uma $\alpha_1 \cdots \alpha_n$ -floresta $\mathcal{F}$ formada pelas árvores $\mathcal{T}_1, \dots, \mathcal{T}_n$. Portanto, a palavra impressa pelo algoritmo no passo k é

$$\alpha_1 \cdots \alpha_n.$$

Digamos que α_j é uma variável, mas que $\alpha_1, \dots, \alpha_{j-1}$ são terminais. Portanto, a árvore mais à esquerda de $\mathcal{F}$, cuja raiz é rotulada por uma variável, é $\mathcal{T}_j$. Assim, ao executar o $(k+1)$ -ésimo passo do algoritmo deveremos remover a raiz de $\mathcal{T}_j$. Mas ao fazer isto estamos substituindo $\mathcal{T}_j$ em $\mathcal{F}$ por uma $\beta_1 \cdots \beta_r$ -floresta, onde $\alpha_j \rightarrow \beta_1 \cdots \beta_r$ é a regra associada à raiz de $\mathcal{T}_j$. Ao final deste passo, o algoritmo terá impresso

$$\alpha_1 \cdots \alpha_{j-1} \beta_1 \cdots \beta_r \alpha_{j+1} \cdots \alpha_n.$$

Entretanto, α_j era a variável mais à esquerda de $\alpha_1 \cdots \alpha_n$, e

$$\alpha_1 \cdots \alpha_{j-1} \alpha_j \alpha_{j+1} \cdots \alpha_n \Rightarrow \alpha_1 \cdots \alpha_{j-1} \beta_1 \cdots \beta_r \alpha_{j+1} \cdots \alpha_n.$$

Como o algoritmo começa imprimindo X , podemos concluir que produz uma derivação mais à esquerda em $\mathcal{G}$, a partir de X . Falta apenas mostrar que o que é derivado é mesmo $c(\mathcal{T})$. Contudo, a colheita das florestas a cada passo da aplicação do algoritmo é sempre a mesma. Além disso, as árvores gramaticais que constituem a floresta no momento que o algoritmo termina têm suas raízes indexadas por terminais. Como o algoritmo não apaga nenhum vértice indexado por terminal diferente de ϵ , concluímos que a concatenação das raízes das árvores no momento em que o algoritmo pára é $c(\mathcal{T})$, como desejávamos.

4. Equivalência entre árvores e derivações

Passemos à recíproca da questão considerada na seção 3. Mais precisamente, queremos mostrar que se $\mathcal{G}$ é uma gramática livre de contexto então toda palavra que tem uma derivação em $\mathcal{G}$ é colheita de alguma árvore de derivação de $\mathcal{G}$.

Para resolver este problema usando um algoritmo precisaríamos inventar uma receita recursiva para construir uma árvore de derivação a partir de uma derivação qualquer em $\mathcal{G}$. Isto é possível, mas o algoritmo resultante não é tão enxuto quanto o anterior. Por isso vamos optar por dar uma demonstração indireta, por indução.

PROPOSIÇÃO 10.2. *Seja X uma variável da gramática livre de contexto $\mathcal{G}$. Se existe uma derivação $X \Rightarrow^* w$, então w é colheita de uma X -árvore em $\mathcal{G}$.*

DEMONSTRAÇÃO. Suponhamos que a gramática livre de contexto $\mathcal{G}$ tem terminais T e variáveis V . A proposição será provada por indução no número p de passos de uma derivação $X \Rightarrow^p w$.

A base da indução consiste em supor que existe uma derivação $X \Rightarrow w$ de apenas um passo. Mas isto só pode ocorrer se existem terminais $t_1, \dots, t_n$ e uma regra da forma $X \rightarrow t_1 \cdots t_n$. Assim, w é a colheita de uma X -árvore que tem a raiz rotulada por X e as folhas por $t_1, \dots, t_n$, o que prova a base da indução.

A hipótese de indução afirma que, se $Y \in \mathcal{V}$ e se existe uma derivação

$$Y \Rightarrow^p u \in \mathcal{T}^*,$$

então u é colheita de uma Y -árvore de $\mathcal{G}$. Digamos, agora, que $w \in \mathcal{T}^*$ pode ser derivado a partir de $X \in V$ em $p + 1$ passos. O primeiro passo desta derivação será da forma $X \Rightarrow v_1 v_2 \cdots v_n$, onde $v_1, \dots, v_n \in T \cup V$ e $X \rightarrow v_1 v_2 \cdots v_n$ é uma regra de $\mathcal{G}$. A derivação continua com cada v_i deflagrando uma derivação da forma $v_i \Rightarrow^* u_i$ onde $u_1 \cdots u_n = w$. Como cada uma destas derivações tem comprimento menor ou igual a p , segue da hipótese de indução que existem v_i -árvores $\mathcal{T}_i$ com colheita u_i , para cada $i = 1, \dots, n$. Mas $\mathcal{T}_1, \dots, \mathcal{T}_n$ é uma $v_1 \cdots v_n$ -floresta, e a primeira regra utilizada na derivação de w foi $X \rightarrow v_1 \cdots v_n$. Portanto, pela definição de árvore gramatical, podemos colar as raízes desta floresta de modo a obter uma X -árvore cuja colheita é

$$c(\mathcal{T}_1) \cdots c(\mathcal{T}_n) = u_1 \cdots u_n = w,$$

o que prova o passo de indução. O resultado desejado segue pelo princípio de indução finita.

Só nos resta reunir, para referência futura, tudo o que aprendemos sobre a relação entre derivações e árvores gramaticais em um único teorema. Antes de enunciar o teorema, porém, observe que tudo o que fizemos usando derivações mais à esquerda vale igualmente para derivações mais à direita.

TEOREMA 10.3. *Seja $\mathcal{G}$ uma gramática livre de contexto e $w \in L(\mathcal{G})$. Então:*

- (1) *existe uma árvore de derivação cuja colheita é w ;*
- (2) *a cada árvore de derivação cuja colheita é w corresponde uma única derivação mais à esquerda de w ;*
- (3) *a cada árvore de derivação cuja colheita é w corresponde uma única derivação mais à direita de w .*

Temos que (1) segue da proposição acima e que (2) é consequência do algoritmo da seção 3. Já (3) segue de uma modificação óbvia deste mesmo algoritmo. A importância deste teorema ficará clara nos próximos capítulos.

5. Ambigüidade

Como vimos na seção 1, as árvores gramaticais são usadas na gramática da língua portuguesa para representar a análise sintática de uma frase em um diagrama. Portanto, têm como finalidade ajudar-nos a interpretar corretamente uma frase.

Contudo, é preciso não esquecer que é possível escrever sentenças gramaticalmente corretas em português que, apesar disso, admitem duas interpretações distintas. Por exemplo, a frase

a seguir veio uma mãe com uma criança empurrando
um carrinho,

pode significar que a mãe empurrava um carrinho com a criança dentro; ou que a mãe vinha com uma criança que brincava com um carrinho. Ambos os sentidos são admissíveis, mas a função sintática das palavras num, e noutro caso, é diferente. No primeiro caso o sujeito que corresponde ao verbo *empurrar* é *mãe*, no segundo caso é *criança*. Assim, teríamos que escolher entre duas árvores gramaticais diferentes para representar esta frase, cada uma correspondendo a um dos sentidos acima.

No caso de uma gramática livre de contexto $\mathcal{G}$, formalizamos esta noção de dupla interpretação na seguinte definição. A gramática $\mathcal{G}$ é *ambígua* se existe uma palavra $w \in L(\mathcal{G})$ que admite duas árvores de derivação distintas em $\mathcal{G}$.

É bom chamar a atenção para o fato de que nem toda frase com duplo sentido em português se encaixa na definição acima. Por exemplo,

“a galinha está pronta para comer” tem dois significados diferentes, mas em ambos as várias palavras da frase têm a mesma função sintática. Assim, não importa o significado que você dê à frase, o sujeito é sempre ‘a galinha’.

Frases que admitem significados diferentes são uma fonte inesgotável de humor; mas para o compilador de uma linguagem de programação uma instrução que admite duas interpretações distintas pode ser um desastre. Voltemos por um momento à gramática $\mathcal{G}_{exp}$. Na figura abaixo, à direita, reproduzimos a árvore de derivação de $id + id * id$ que já havíamos esboçado na seção 2. Uma árvore de derivação diferente para a mesma expressão pode ser encontrada à esquerda, na mesma figura.



FIGURA 2. Duas árvores e uma mesma colheita

A existência desta segunda árvore de derivação significa que, se um computador estiver usando a gramática $\mathcal{G}_{exp}$, ele não saberá como distinguir a precedência correta entre os operadores $+$ e $*$. Dito de outra maneira, esta gramática não permite determinar que, quando nos deparmos com $id + id * id$ devemos primeiro efetuar a multiplicação e só depois somar o produto obtido com a outra parcela da soma.

Uma saída possível é tentar inventar uma outra gramática que gere a mesma linguagem, mas que não seja ambígua. A idéia básica consiste em introduzir novas variáveis e novas regras que forcem a precedência correta. Para fazer isto, compare novamente as duas árvores gramaticais com colheita $id + id * id$ da figura 2.

Observe que, na árvore da direita, $id * id$ é obtida a partir da expressão $E * E$, e os vértices que correspondem a estes três rótulos são todos irmãos. Por outro lado, o único vértice do qual descendem todos os símbolos da expressão $id + id * id$ é a própria raiz. Portanto, interpretando a expressão $id + id * id$ conforme a árvore da direita, teremos primeiro que calcular o produto, e só depois a soma. Interpretando a mesma expressão de acordo com a árvore da esquerda, vemos que neste caso a adição $id + id$ é efetuada antes, e o resultado, então, multiplicado

por *id*. Isto é, a árvore que dá a interpretação correta da precedência dos operadores é a que está à direita da figura 2.

Assim, precisamos construir uma nova gramática na qual uma árvore como a que está à direita da figura possa ser construída, mas não uma como a da esquerda.

A estratégia consiste em introduzir novas variáveis de modo a forçar a precedência correta dos operadores. Faremos isto deixando a variável *E* controlar as adições, e criando uma nova variável *F* (de *fator*) para controlar a multiplicação. O conjunto de regras resultante é o seguinte:

$$E \rightarrow E + F$$

$$E \rightarrow F$$

$$F \rightarrow F * F$$

$$F \rightarrow (E)$$

$$F \rightarrow \text{id}$$

Não há nada de mais a comentar sobre a primeira e a terceira regras; e a segunda apenas permite passar de somas a multiplicações. A regra que realmente faz a diferença é a quarta. Ela nos diz que, após efetuar uma multiplicação, só podemos voltar à variável *E* (que controla as somas) colocando os parêntesis.

Embora esta nova gramática não permita a construção de uma árvore como a da esquerda na figura 2, ainda assim ela é ambígua. Por exemplo, as árvores da figura 3 estão de acordo com a nova gramática mas, apesar de diferentes, têm ambas colheita $\text{id} * \text{id} * \text{id}$.

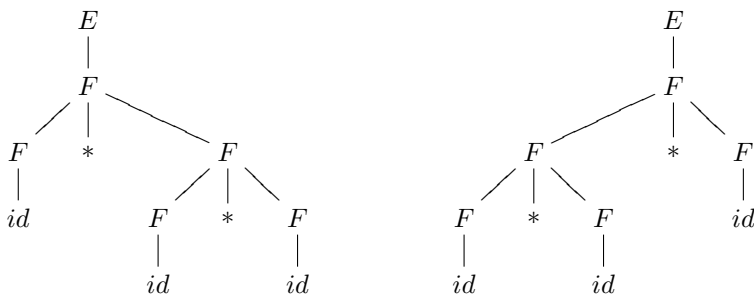


FIGURA 3. Outras duas árvores com mesma colheita

A saída é introduzir mais uma variável, que vamos chamar de *T* (para *termo*). A nova gramática, que chamaremos de $\mathcal{G}'_{exp}$, tem variáveis

E , T e F , símbolo inicial E , e as seguintes regras:

$$\begin{aligned} E &\rightarrow E + T \quad | \quad T \\ T &\rightarrow T * F \quad | \quad F \\ F &\rightarrow (E) \quad | \quad \text{id} \end{aligned}$$

A gramática $\mathcal{G}'_{exp}$ não é ambígua, mas provar isto não é fácil, e não faremos os detalhes aqui.

Ainda há um ponto que precisa ser esclarecido. A discussão anterior pode ter deixado a impressão de que, para estabelecer a precedência correta entre os operadores aritméticos, basta eliminar a ambigüidade da gramática. Mas isto não é verdade. Por exemplo, a gramática $\mathcal{G}''_{exp}$ cujas regras são:

$$\begin{aligned} S &\rightarrow E \\ E &\rightarrow (E)R \quad | \quad VR \\ R &\rightarrow +E \quad | \quad *E \quad | \quad A \\ V &\rightarrow \text{id} \\ A &\rightarrow \epsilon \end{aligned}$$

gera L_{exp} . Além disso, não é difícil provar que esta gramática não pode ser ambígua. Isto decorre dos seguintes fatos:

- não há mais de duas regras em $\mathcal{G}''_{exp}$ cujo lado esquerdo seja ocupado por uma mesma variável;
- se há duas regras a partir de uma mesma variável, o lado direito de uma começa com uma variável, e o da outra com um terminal;
- cada terminal só aparece uma vez como prefixo do lado direito de alguma regra de $\mathcal{G}''_{exp}$.

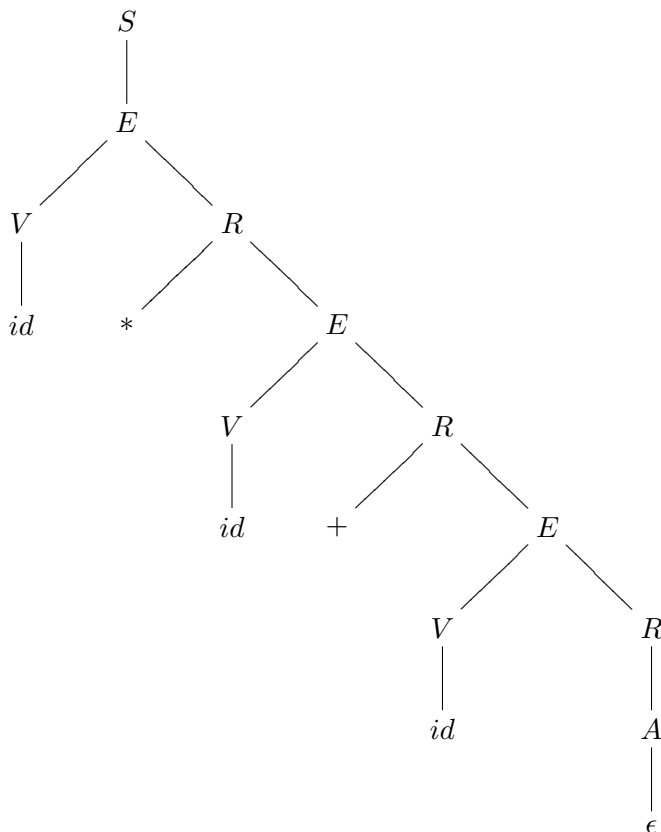
Imagine, então, que estamos tentando calcular uma derivação à esquerda em $\mathcal{G}''_{exp}$ para uma dada palavra $w \in L_{exp}$. Digamos que, isolando o n -ésimo passo da derivação, obtivemos o seguinte

$$S \Rightarrow^* uEv \Rightarrow w,$$

onde u só contém terminais, mas v pode conter terminais e variáveis. Precisamos decidir qual a regra a ser aplicada a seguir. Neste exemplo, a variável mais à esquerda no n -ésimo passo da derivação é E . Assim, há duas regras que podemos aplicar. Para saber qual das duas será escolhida voltamos nossa atenção para a palavra w que está sendo derivada. É claro que w tem u como prefixo; digamos que o terminal seguinte a u em w seja σ . Temos então duas possibilidades. Se $\sigma = ($ então a regra

a ser aplicada tem que ser $E \rightarrow (E)R$. Por outro lado, se $\sigma \neq ($ então só nos resta a possibilidade de aplicar $E \rightarrow VR$. Assim, a regra a ser aplicada neste passo está completamente determinada pela variável mais à esquerda presente, e pela sequência de terminais da palavra que está sendo derivada.

Entretanto, apesar de $\mathcal{G}''_{exp}$ não ser ambígua, a árvore de derivação de $id + id * id$ em $\mathcal{G}''_{exp}$ não apresenta a precedência desejada entre os operadores, como mostra a figura abaixo.



Voltaremos a discutir $\mathcal{G}''_{exp}$ quando tratarmos de autômatos de pilha determinísticos.

6. Removendo ambigüidade

O que fizemos na seção anterior parece indicar que, ao se deparar com uma gramática ambígua, tudo o que temos que fazer é adicionar algumas variáveis e alterar um pouco as regras de maneira a remover a

ambigüidade. É verdade que criar novas variáveis e regras para remover ambigüidade pode não ser muito fácil, e é bom não esquecer que não chegamos a provar que a gramática $\mathcal{G}'_{exp}$ não é ambígua.

Por outro lado, isto parece bem o tipo de problema que poderia ser deixado para um computador fazer, bastaria que descobríssemos o algoritmo. É aí justamente que está o problema. Um computador não consegue sequer detectar que uma linguagem é ambígua. De fato:

não pode existir um algoritmo para determinar se uma dada linguagem livre de contexto é ou não ambígua.

Voltaremos a esta questão em um capítulo posterior.

Infelizmente, as más notícias não acabam aí. Há linguagens livres de contexto que não podem ser geradas por nenhuma gramática livre de contexto que não seja ambígua. Tais linguagens são chamadas de *inerentemente ambíguas*. Note que ambigüidade é uma propriedade da gramática, mas ambigüidade inerente é uma propriedade da própria linguagem.

Um exemplo bastante simples de linguagem inerentemente ambígua é

$$L_{in} = \{a^i b^j c^k : i, j, k \geq 0 \text{ e } i = j \text{ ou } j = k\}.$$

Com isso você descobre porque demos este nome a L_{in} . Não é fácil mostrar que esta linguagem é inerentemente ambígua, e por isso não vamos fazer a demonstração aqui. Os detalhes podem ser encontrados em [1, theorem 7.2.2, p. 236] ou [2, theorem 4.7, p. 100].

Para não encerrar o capítulo num clima pessimista, vamos analisar o problema da ambigüidade para o caso particular das linguagens regulares. Sabemos que estas linguagens podem ser geradas por gramáticas livres de contexto de um tipo bastante especial: as gramáticas lineares à direita. Assim a primeira pergunta é: uma gramática linear à direita pode ser ambígua? A resposta é sim. Por exemplo, considere a gramática com terminal $\{0\}$, que tem S como única variável, e cujas regras são

$$S \rightarrow 0 \quad | \quad 0S \quad | \quad 0^2.$$

A palavra 0^2 tem duas árvores gramaticais distintas que estão esboçadas na abaixo.



Felizmente, no caso de gramáticas lineares à direita é sempre possível remover a ambigüidade. Em outras palavras, não existem linguagens regulares inerentemente ambíguas. Além disso, existe um algoritmo simples que, tendo como entrada uma gramática linear à direita, constrói uma outra que gera a mesma linguagem regular mas não é ambígua. O algoritmo é consequência do seguinte fato:

a gramática linear à direita construída a partir de um autômato finito determinístico pelo algoritmo do capítulo 8 *não* é ambígua.

Para entender porque isto é verdade, suponha que M seja um autômato finito determinístico e $\mathcal{G}$ seja a gramática construída a partir de M pelo algoritmo do capítulo ???. Vimos que as derivações em $\mathcal{G}$ simulam computações em M e vice-versa. Como M é determinístico, só há uma computação possível para cada palavra de $L(M)$. Logo cada palavra de $L(\mathcal{G}) = L(M)$ só tem uma derivação possível. Como $\mathcal{G}$ é linear à direita, toda derivação em $\mathcal{G}$ é mais à esquerda. Logo $\mathcal{G}$ não é ambígua pelo teorema da seção 4.

Diante deste resultado, é claro que, ao receber uma gramática linear à direita $\mathcal{G}$ como entrada, o algoritmo procede da seguinte maneira:

Etapa 1: determina um autômato finito não determinístico M tal que $L(M) = L(\mathcal{G})$;

Etapa 2: determina um autômato finito determinístico M' tal que $L(M) = L(M')$;

Etapa 3: determina uma gramática $\mathcal{G}'$, obtida a partir de M' , e que gera $L(M')$.

Como já vimos, $\mathcal{G}'$ não pode ser uma gramática ambígua.

7. Exercícios

1. Prove, por indução no número de vértices internos, que a colheita de uma árvore gramatical T é igual à palavra obtida concatenando-se os rótulos das folhas de T da esquerda para a direita.
2. Seja $\mathcal{G}$ uma gramática livre de contexto e seja X uma variável de $\mathcal{G}$. Seja w uma palavra somente em terminais e que pode ser derivada em $\mathcal{G}$ a partir de X . Prove, por indução no número de passos de uma derivação de w a partir de X que existe uma X -árvore em $\mathcal{G}$ cuja colheita é w .
3. Considere a gramática não ambígua G'_{exp} que gera as expressões aritméticas.

- (a) Esboce as árvores de derivação de $\text{id} + (\text{id} + \text{id}) * \text{id}$ e de $(\text{id} * \text{id} + \text{id} * \text{id})$
- (b) Dê uma derivação à esquerda e uma derivação à direita da expressão $(\text{id} * \text{id} + \text{id} * \text{id})$.
4. Repita o exercício anterior para a gramática $\mathcal{G}''_{exp}$.
5. Descreva detalhadamente um algoritmo que, tendo como entrada a derivação de uma palavra w em uma gramática livre de contexto, constrói uma árvore gramatical cuja colheita é w . O principal problema consiste em, tendo dois passos consecutivos da derivação, determinar qual a regra que foi aplicada.
6. Prove que a gramática $\mathcal{G}''_{exp}$ não é ambígua.
SUGESTÃO: Use indução no número de passos de uma derivação à esquerda.
7. Mostre que a gramática cujas regras são

$$S \rightarrow 1A \mid 0B$$

$$A \rightarrow 0 \mid 0S \mid 1AA$$

$$B \rightarrow 1 \mid 1S \mid 0BB$$

é ambígua.

8. Seja G a gramática linear à direita com terminais $\{0, 1\}$, variáveis $\{X, Y, Z, W\}$ e símbolo inicial S , cujas regras são dadas por

$$S \rightarrow 0X$$

$$X \rightarrow 10Y \mid 1Z$$

$$Z \rightarrow 01W \mid 1$$

$$Y \rightarrow 1 \mid 0$$

$$W \rightarrow \epsilon$$

- (a) Mostre que G é ambígua.
- (b) Use o algoritmo descrito em 6 para construir uma gramática linear à direita G' não ambígua que gere $L(G)$.

Linguagens que não são livres de contexto

Neste capítulo, finalmente, confrontamos a inevitável pergunta: como provar que uma dada linguagem não é livre de contexto? A estratégia é muito semelhante à adotada para linguagens regulares, apesar do lema do bombeamento resultante ser um pouco mais difícil de aplicar na prática.

1. Introdução

Já conhecemos muitas linguagens livres de contexto, mas ainda não temos nenhum exemplo de uma linguagem que não seja deste tipo. Quer dizer, já dissemos no capítulo 9 que a linguagem

$$L_{abc} = \{a^n b^n c^n : n \geq 0\}$$

não é livre de contexto, mais ainda não provamos isto. É claro que isto não é satisfatório. O problema é que nada podemos concluir do simples fato de não termos sido capazes de inventar uma gramática livre de contexto que gere esta linguagem. Talvez a gramática seja muito complicada, ou quem sabe foi só falta de inspiração.

Mas como será possível provar que não existe nenhuma gramática livre de contexto que gere uma dada linguagem? A estratégia é a mesma adotada para o caso de linguagens regulares. Isto é, provaremos que toda linguagem livre de contexto satisfaz uma propriedade de bombeamento. Portanto, uma linguagem que *não* satisfizer esta propriedade *não* pode ser livre de contexto. Começamos com um resultado relativo a árvores que será necessário na demonstração do lema do bombeamento.

Como no capítulo anterior, consideraremos apenas árvores enraizadas. Dizemos que uma árvore é *m-ária* se cada vértice tem, no máximo,

m filhos. Desejamos relacionar o número de folhas de uma árvore m -ária com a altura desta árvore. Lembre-se que a *altura* de uma árvore é o mais longo caminho entre a raiz e alguma de suas folhas.

Quando todos os vértices internos da árvore têm exatamente m filhos, a árvore m -ária é *completa*. Seja $f(h)$ o número de folhas de uma árvore m -ária completa de altura h . Como a árvore m -ária completa é aquela que tem o maior número possível de folhas, o problema estará resolvido se formos capazes de encontrar uma fórmula para $f(h)$ em função de h e m . Faremos isto determinando uma relação de recorrência para $f(h)$ e resolvendo-a.

Para começar, se a árvore tem altura zero, então consiste apenas de um vértice. Neste caso há apenas uma folha, de modo que $f(0) = 1$. Para estabelecer a relação de recorrência podemos imaginar que $\mathcal{T}$ é uma árvore m -ária completa de altura h . É claro que, removendo todas as folhas de $\mathcal{T}$, obtemos uma árvore m -ária completa $\mathcal{T}'$ de altura $h - 1$. Para reconstruir $\mathcal{T}$ a partir de $\mathcal{T}'$ precisamos repor as folhas. Fazemos isso dando m -filhos a cada folha de $\mathcal{T}'$. Como $\mathcal{T}'$ tem $f(h - 1)$ folhas, obtemos

$$f(h) = mf(h - 1).$$

Assim,

$$f(h) = mf(h - 1) = m^2 f(h - 2) = \dots = m^h f(0) = m^h.$$

Portanto, $f(h) = m^h$ é a fórmula desejada.

Para estabelecer a relação entre este resultado e o lema do bombeamento precisamos de uma definição. Seja $\mathcal{G}$ uma linguagem livre de contexto. A *amplitude* $\alpha(\mathcal{G})$ de uma gramática livre de contexto $\mathcal{G}$ é o comprimento máximo das palavras que aparecem à direita de uma seta em uma regra de $\mathcal{G}$. Por exemplo, para as gramáticas $\mathcal{G}_{exp}$ e $\mathcal{G}_{exp}''$ definidas no capítulo anterior, temos $\alpha(\mathcal{G}_{exp}) = 3$ e $\alpha(\mathcal{G}_{exp}'') = 4$.

Se uma gramática livre de contexto tem amplitude α , então todas as suas árvores gramaticais são α -árias. A fórmula para o número de folhas de uma árvore α -ária completa nos dá então seguinte lema.

LEMA 11.1. *Seja $\mathcal{G}$ uma linguagem livre de contexto. Se X é uma variável de $\mathcal{G}$ e se w é colheita de uma X -árvore de $\mathcal{G}$ de altura h então*

$$|w| \leq \alpha(\mathcal{G})^h.$$

2. Lema do bombeamento

Antes de enunciar e provar o lema do bombeamento de maneira formal, vamos considerar o seu funcionamento de maneira informal.

Suponhamos que $\mathcal{G}$ é uma gramática livre de contexto que gera uma linguagem infinita. Segundo o lema da seção 1, quanto maior o comprimento da colheita de uma árvore gramatical maior tem que ser a sua altura. Portanto, se o comprimento da colheita de uma árvore de derivação $\mathcal{T}$ é suficientemente grande, então o caminho mais longo entre a raiz e alguma folha conterá mais vértices interiores do que há variáveis na gramática. Em particular, haverá dois vértices diferentes em $\mathcal{T}$ cujos rótulos são iguais. Vamos chamar de ν_1 e ν_2 estes vértices, e de A a variável que os rotula, como na figura 1.

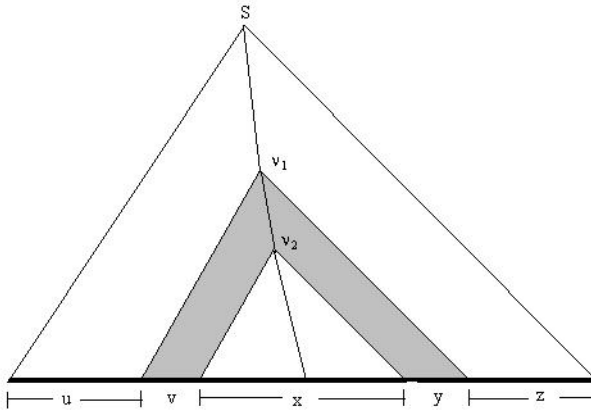


FIGURA 1. Decompondo a palavra para bombear

Observe que as regras associadas a ν_1 e ν_2 têm ambas A do seu lado esquerdo. Mas isto significa que podemos construir a partir de $\mathcal{T}$ uma nova árvore $\mathcal{T}'$ da seguinte maneira. Comece construindo $\mathcal{T}'$ exatamente como $\mathcal{T}$ até chegar a ν_2 . Como ν_2 é rotulado pela mesma variável que ν_1 , podemos construir a partir dele a mesma A -árvore que estava associada a ν_1 em $\mathcal{T}$, como mostra a figura 2. Note que os trechos da colheita da árvore $\mathcal{T}$ da figura 1 marcados como v e y aparecem repetidos em $\mathcal{T}'$. Como $\mathcal{T}'$ é uma árvore de derivação em $\mathcal{G}$, sua colheita é um elemento de $L(\mathcal{G})$ no qual os trechos x e y de $c(\mathcal{T})$ aparecem bombeados. Naturalmente o processo acima pode ser repetido quantas vezes quisermos, de modo que podemos bombear estes trechos qualquer número de vezes.

Para que esta propriedade do bombeamento possa ser usada para provar que uma linguagem não é livre de contexto precisamos formulá-la de maneira mais precisa. Além disso, como no caso do lema correspondente para linguagens regulares, incluiremos algumas condições

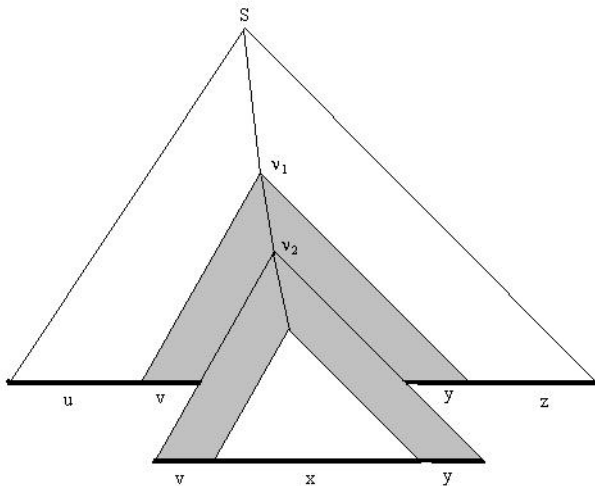


FIGURA 2. Bombeando uma vez

técnicas extras que reduzem o número de possíveis decomposições da palavra a ser bombeada que precisamos considerar. De fato, há várias versões diferentes do lema do bombeamento para linguagens livres de contexto. A que apresentamos aqui não é a mais forte, mas é suficiente para cobrir muitos dos exemplos mais simples. Versões mais sofisticadas são discutidas em [2, Chapter 6, p. 125].

LEMA DO BOMBEAMENTO. *Seja $\mathcal{G}$ uma gramática livre de contexto. Existe um número inteiro ρ , que depende de $\mathcal{G}$, tal que, se $w \in L(\mathcal{G})$ e $|w| \geq \rho$, então existe uma decomposição de w na forma $w = uvxyz$, onde*

- (1) $vy \neq \epsilon$;
- (2) $|vxy| \leq \rho$;
- (3) $uv^nxy^n z \in L(\mathcal{G})$, para todo $n \geq 0$.

DEMONSTRAÇÃO. Suponhamos que $\mathcal{G}$ tem k variáveis; neste caso escolheremos $\rho = \alpha(\mathcal{G})^{k+1}$. Seja $w \in L(\mathcal{G})$ uma palavra com comprimento maior que ρ . Já sabemos, pelo teorema da seção 4 do capítulo 10, que têm que existir árvores de derivação com colheita w . Entre todas estas árvores escolha uma, que chamaremos de $\mathcal{T}$, que satisfaça a seguinte propriedade:

Hipótese 1: $\mathcal{T}$ tem o menor número possível de folhas entre todas as árvores de derivação de colheita w em $\mathcal{G}$.

Como a colheita de $\mathcal{T}$ tem comprimento maior que $\rho = \alpha(\mathcal{G})^{k+1}$, segue do lema da seção 1 que $\mathcal{T}$ tem altura pelo menos $k+1$. Chamando de C o mais longo caminho em $\mathcal{T}$ que vai de sua raiz a uma folha, concluímos que C tem, pelo menos, $k+1$ arestas. Logo C tem, no mínimo, $k+2$ vértices. Como só pode haver uma folha num tal caminho, então C tem $k+1$ vértices interiores. Contudo k é o número de variáveis da gramática, e cada vértice de C está rotulado por uma variável. Portanto, pelo princípio da casa do pombo, há dois vértices *diferentes* em C rotulados pela mesma variável. Entre todos os pares de vértices de C rotulados pela mesma variável, escolha aquele que satisfaz a seguinte propriedade:

Hipótese 2: o vértice ν_1 precede o vértice ν_2 e todos os vértices de C entre ν_1 e a folha são rotulados por variáveis distintas.

Lembre-se que as árvores que estamos considerando são grafos orientados. Portanto, C é um caminho orientado; logo faz sentido dizer, de dois vértices de C , que um precede o outro.

Seja A a variável que rotula ν_1 e ν_2 . Temos então duas A -árvores: $\mathcal{T}_1$, com raiz em ν_1 , e $\mathcal{T}_2$ com raiz em ν_2 . Denotaremos por x a colheita de $\mathcal{T}_2$. Observe que, como ν_1 precede ν_2 ao longo de C , então x é uma subpalavra da colheita de $\mathcal{T}_1$. Assim, podemos decompor a colheita de $\mathcal{T}_1$ na forma vxy . A relação entre estas árvores e suas colheitas é ilustrada na figura 1.

Entretanto, pela hipótese 2, o caminho (ao longo de C) que vai de ν_1 à folha tem, no máximo, $k+2$ vértices (contando com a folha, que é rotulada por um terminal!). Além disso, como C é o mais longo caminho que vai da raiz de $\mathcal{T}$ a uma folha, o trecho de C que começa em ν_1 é o mais longo caminho em $\mathcal{T}_1$ entre sua raiz (que é ν_1) e uma folha. Portanto, $\mathcal{T}_1$ tem altura no máximo $k+1$. Concluímos, utilizando novamente o lema da seção 1 que, como vxy é a colheita de $\mathcal{T}_1$, então

$$|vxy| \leq \alpha(\mathcal{G})^{k+1} = \rho.$$

Isto prova (2) do enunciado do lema.

Considere agora o que acontece na construção de $\mathcal{T}$ quando chegamos a ν_2 . Este vértice é rotulado pela variável A e a ele está associada uma regra que tem A do lado esquerdo da seta. Mas suponha que, chegados a ν_2 , decidimos aplicar a mesma regra que aplicamos quando chegamos a ν_1 . Podemos fazer isto porque esta também é uma regra que tem A do lado esquerdo. Se continuarmos, vértice a vértice, copiando o trecho da árvore $\mathcal{T}$ hachurado na figura, teremos uma nova árvore gramatical em $\mathcal{G}$, cuja colheita é uv^2xy^2z . Se repetirmos este procedimento

n vezes, obteremos uma árvore cuja colheita é $wv^nx y^n z$. Isto prova (3) quando $n > 0$.

Por outro lado, ao chegar ao vértice ν_1 , também podemos usar a regra associada a ν_2 e continuar a construir a árvore como se fosse $\mathcal{T}_2$. Neste caso obteremos uma árvore $\mathcal{T}_0$ com menos vértices que $\mathcal{T}$ e com colheita uxz , que corresponde a tomar $n = 0$ em (3).

Só nos resta mostrar que $vy \neq \epsilon$. Mas se vy fosse igual a ϵ então a árvore $\mathcal{T}_0$, construída no parágrafo anterior, teria colheita igual a w , e menos folhas que $\mathcal{T}$, o que contradiz a hipótese 1. Portanto, $vy \neq \epsilon$ e provamos (1), concluindo assim a demonstração do lema do bombeamento.

3. Exemplos

Veremos a seguir que várias das linguagens que já encontramos anteriormente, e outras que ainda vamos encontrar à frente, não são livres de contexto. Antes, porém, precisamos discutir como o lema do bombeamento é utilizado para provar que uma linguagem não é livre de contexto.

Digamos que L é uma linguagem que você suspeita não ser livre de contexto. Para aplicar o lema do bombeamento a L usamos a mesma estratégia já utilizada no caso de linguagens regulares. Assim,

- (1) suporemos, por contradição, que L é gerada por uma gramática livre de contexto;
- (2) escolheremos uma palavra $w \in L$ de comprimento maior que ρ ;
- (3) mostraremos que não há *nenhuma maneira possível* de decompor w na forma do lema do bombeamento, de modo que w tenha subpalavras bombeáveis.

Com isto podemos concluir que L não é livre de contexto.

É claro que, se o seu palpite estiver errado e L for livre de contexto, então você não chegará a nenhuma contradição. Por outro lado, o fato de uma contradição não ter sido obtida *não* significa que L é livre de contexto.

Como no caso das linguagens regulares, a escolha da palavra em (2) depende de ρ , uma variável inteira positiva. Além disso, escolher w de maneira a obter facilmente uma contradição envolve uma certa dose de tentativa e erro. Finalmente, por causa da maneira mais complicada de decompor w , podemos ter vários casos a analisar antes de esgotar todas as possibilidades. Vejamos alguns exemplos.

Exemplo 1. Já havíamos dito no capítulo 7 que a linguagem

$$L_{abc} = \{a^n b^n c^n : n \geq 0\}$$

não é livre de contexto. Temos, agora, as ferramentas necessárias para provar que isto é verdade.

Suponha, por contradição, que L_{abc} é livre de contexto. Pelo lema do bombeamento existe um inteiro positivo ρ tal que, se $n > \rho$, então é possível decompor $w = a^n b^n c^n$ na forma $w = uvxyz$, onde:

- (1) $vy \neq \epsilon$;
- (2) $|vxy| \leq \rho$;
- (3) $uv^n xy^n z \in L_{abc}$ para todo $n \geq 0$.

Como $n > \rho$ mas $|vxy| \leq \rho$, temos que vxy não pode conter, ao mesmo tempo, as , bs e cs . Digamos que vxy só contenha as e bs . Neste caso, nem v , nem y , podem conter c , mas vy tem que conter pelo menos um a ou um b . Assim, quando $n > 1$ o número de as ou bs em $uv^n xy^n z$ tem que ser maior que n , ao passo que o número de cs não foi alterado, e continua sendo n . O caso em que vxy só contém bs e cs pode ser tratado de maneira análoga. Portanto, $uv^n xy^n z \notin L_{abc}$ o que contradiz o lema do bombeamento. Concluimos que, de fato, L não é uma linguagem livre de contexto.

Exemplo 2. No capítulo 3 vimos que a linguagem

$$L_{\text{primos}} = \{0^p : p \text{ é um primo positivo}\},$$

não é regular. Como já sabemos que há linguagens livres de contexto que não são regulares, faz sentido perguntar se esta linguagem é livre de contexto. A resposta é não.

Suponhamos, por contradição, que L_{primos} seja livre de contexto. Então, de acordo com o lema do bombeamento, deve existir um inteiro positivo ρ tal que se p é um primo maior que ρ , então podemos decompor 0^p na forma $0^p = uvxyz$, onde

- (1) $vy \neq \epsilon$;
- (2) $|vxy| \leq \rho$;
- (3) $uv^n xy^n z \in L_{\text{primos}}$ para todo $n \geq 0$.

Mas u , v , x , y e z são todas palavras em 0^* . Logo existem inteiros $m, r, s, t \geq 0$ tais que

$$u = 0^m, v = 0^r, x = 0^s, y = 0^t \text{ e } z = 0^{p-r-s-t}.$$

Além disso, segue de (1) que $r + t > 0$, e de (3) que

$$uv^n xy^n z = 0^m (0^r)^n 0^s (0^t)^n 0^{p-r-s-t} = 0^{p+(r+t)(n-1)}$$

pertence a L_{primos} para todo $n > 0$. Mas, para que esta última afirmação seja verdadeira, os números $p + (r + t)(n - 1)$ têm que ser primos para todo $n \geq 0$. Tomando $n = p + 1$ temos que

$$p + (r + t)(n - 1) = p(1 + r + t),$$

é composto, pois $r + t > 0$; uma contradição. Portanto, L_{primos} não é uma linguagem livre de contexto.

Exemplo 3. Considere, agora, a linguagem

$$L_{rr} = \{rr : s \in (0 \cup 1)^*\}.$$

Já vimos no capítulo 3 que esta linguagem não é regular, queremos provar que também não é livre de contexto.

Suponhamos, por contradição, que L_{rr} é livre de contexto. Aproveitando uma idéia já usada no capítulo 3, escolhemos $w = 0^m 10^m 1$ como nossa primeira tentativa. Entretanto, escolhendo $\rho = 3$ e decompondo $0^m 10^m 1$ na forma

$$u = 0^{m-1}, v = 0, x = 1, y = 0 \quad \text{e} \quad z = 0^{m-1} 1,$$

verificamos que todas as condições do lema do bombeamento são satisfeitas. Portanto, não é possível chegar a uma contradição escolhendo $r = 0^m 1$.

A saída é escolher uma palavra mais complexa. Observe que, no exemplo acima, a decomposição proposta não funcionaria se o número de 1s também dependesse de m . Isto sugere que devemos escolher $r = 0^m 1^m$.

Supondo, por contradição, que L_{rr} é livre de contexto e escolhendo $w = 0^m 1^m 0^m 1^m$ temos, pelo lema do bombeamento, que se $m \geq \rho$ então w pode ser decomposta na forma

$$0^m 1^m 0^m 1^m = uvxyz$$

de modo que as condições (1), (2) e (3) do lema do bombeamento sejam satisfeitas. Há dois casos a considerar.

O primeiro caso consiste em supor que vxy é uma subpalavra do primeiro $0^m 1^m$. Escrevendo $0^m 1^m = uvxyz'$, onde $z = z' 0^m 1^m$, temos que

$$2m < |uv^2xy^2z'| \leq 2m + \rho.$$

Isto significa que yz' foi deslocado $|vxy|$ casas para à direita. Contudo,

$$|vxy| \leq \rho \leq m$$

e vxy^2z' acaba em m uns. Como o segundo $0^m 1^m$ não foi bombeado, temos que o símbolo seguinte ao meio da palavra é 1. Em particular, se $uv^2xy^2z = rr$ então o segundo r começa com 1 e o primeiro com 0, o

que é uma contradição. O caso em que vxy é subpalavra do segundo $0^m 1^m$ pode ser tratado de maneira análoga.

Finalmente, resta-nos supor que vxy inclui o meio da palavra. Neste caso, vxy tem que ser subpalavra de $1^m 0^m$, já que $|vxy| \leq \rho \leq m$. Assim, ou v contém algum 1 ou y algum 0. Removendo-os, concluímos que $uxz = 0^m 1^s 0^t 1^m$, onde s ou t (ou ambos) são menores que m . Entretanto, se $0^m 1^s 0^t 1^m = rr$ então r começa em 0 e acaba em 1, de forma que precisamos ter $s = m = t$, o que é uma contradição. Concluímos que L_{rr} não é livre de contexto.

Exemplo 4. Vimos na seção 4 do capítulo 7 que a união, concatenação e estrela de linguagens livres de contexto também é livre de contexto. Resta-nos analisar o que acontece com a intersecção e o complemento de linguagens livres de contexto.

Considere, por exemplo as linguagens

$$L_1 = \{a^i b^j c^k : i, j, k \geq 0 \text{ e } i = j\}, \text{ e}$$

$$L_2 = \{a^i b^j c^k : i, j, k \geq 0 \text{ e } j = k\}.$$

Já vimos no capítulo 7 que estas duas linguagens são livres de contexto. Entretanto,

$$L_1 \cap L_2 = \{a^i b^j c^k : i = j \text{ e } j = k\}.$$

Em outras palavras, $L_1 \cap L_2 = L_{abc}$. Porém, mostramos no exemplo 1 acima que esta linguagem não é livre de contexto. Assim, a intersecção de duas linguagens livres de contexto pode não ser livre de contexto.

E quanto ao complemento? Já sabemos do capítulo 5, que existe uma maneira de relacionar o complemento e a intersecção de duas linguagens através da união. Como a intersecção de linguagens livres de contexto não é necessariamente livre de contexto, seria de esperar que o mesmo valesse para o complemento. Provaremos isto argumentando por contradição.

Suponhamos, por contradição, que o complemento de linguagens livres de contexto seja livre de contexto. Como L_1 e L_2 são livres de contexto, então $\overline{L_1}$ e $\overline{L_2}$ também serão. Mas a união de linguagens livres de contexto é livre de contexto. Portanto, $\overline{L_1} \cup \overline{L_2}$ é livre de contexto. Usando mais uma vez a hipótese sobre o complemento, concluímos que

$$\overline{(\overline{L_1} \cup \overline{L_2})} = L_1 \cap L_2,$$

é livre de contexto. Entretanto já vimos que isto é falso neste caso. Portanto, o complemento de linguagens livres de contexto pode não ser livre de contexto.

Note que nossa conclusão diz apenas que a intersecção e o complemento de linguagens livres de contexto *pode não ser* livre de contexto. Isto porque, em muitos casos, estas operações produzem linguagens livres de contexto. Por exemplo, toda linguagem regular é livre de contexto; e já vimos que a intersecção e o complemento de linguagens regulares é regular. Portanto, neste caso estamos intersectando ou tomando o complementar de linguagens livres de contexto e obtendo como resultado uma linguagem do mesmo tipo. É possível aperfeiçoar este resultado, e mostrar que a intersecção de uma linguagem livre de contexto com uma linguagem *regular* é sempre livre de contexto. Mas, para fazer isto, precisamos usar os autômatos de pilha que serão introduzidos no próximo capítulo.

4. Exercícios

1. Mostre que nenhuma das linguagens abaixo é livre de contexto usando o lema do bombeamento.
 - (a) $\{a^{2^n} : n \text{ é primo}\}$;
 - (b) $\{a^{n^2} : n \geq 0\}$;
 - (c) $\{a^n b^n c^r : r \geq n\}$;
 - (d) o conjunto das palavras em $\{a, b, c\}^*$ que têm o mesmo número de as e bs , e cujo número de cs é maior ou igual que o de as ;
 - (e) $\{0^n 1^n 0^n 1^n : n \geq 0\}$;
 - (f) $\{r r r : s \in (0 \cup 1)^*\}$;
 - (g) $\{w c w c w : w \in \{0, 1\}^*\}$;
 - (h) $\{0^{n!} : n \geq 1\}$;
 - (i) $\{0^k 1^k 0^k : k \geq 0\}$;